EXP:1

Bresenhams line Drawing Algorithm for |m|<1.

AIM: To implement Bresenhams line drawing Algorithm for |m|<1 using Python Programming.

PROGRAM:

```
def bresenham_line(x1, y1, x2, y2):
    points = []
    dx = x2 - x1
    dy = y2 - y1
    # practical decision
    p = 2 * dy - dx
    x = x1
    y = y1
    points.append((x, y))
    while x < x2:
        x = x + 1
        if p < 0:
            p = p + 2 * dy
        else:
            pixel
            y = y + 1
            p = p + 2 * dy - 2 * dx
        points.append((x, y))
    return points
```

Enter x_1 : 5

Enter y_1 : 8

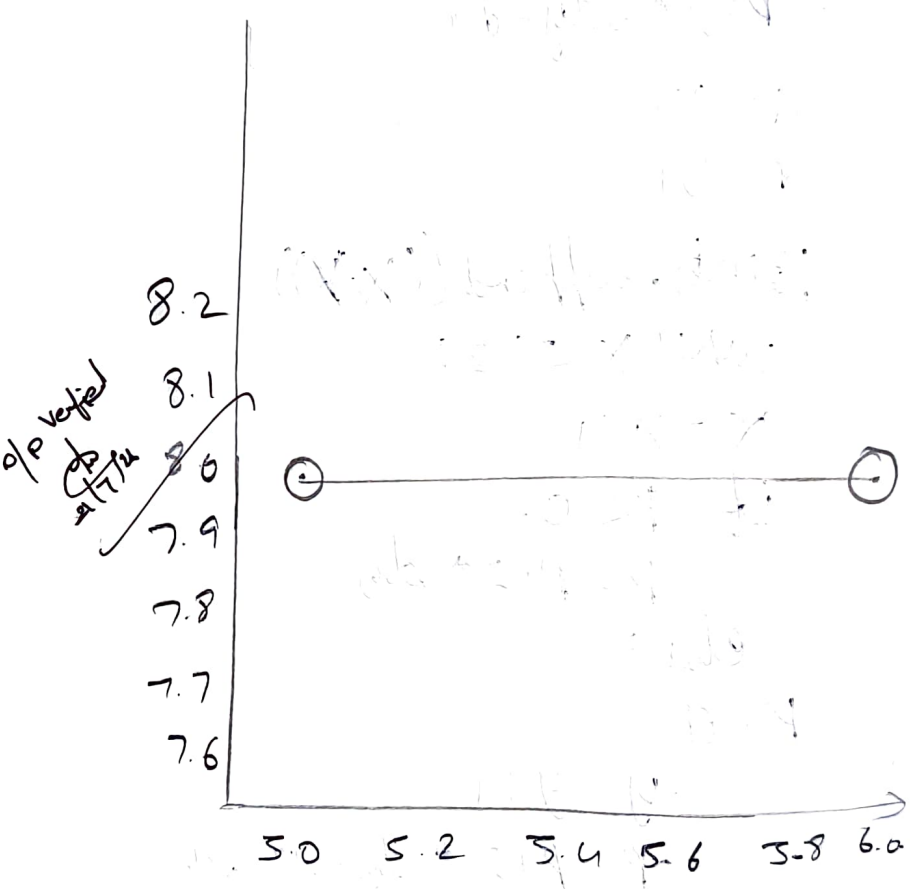
Enter x_2 : 6

Enter y_2 : 2

Points generated

(5, 8)

(6, 8)



```
x1 = int (input ("Enter x1 :"))
y1 = int (input ("Enter y1 :"))
x2 = int (input ("Enter x2 :"))
y2 = int (input ("Enter y2 :"))
points = bresenham_line (x1, y1, x2, y2)
print ("Points generated:")
for p in points:
print (p)
x_values = [p[0] for p in points]
y_values = [p[1] for p in points]
plt.plot (x_values, y_values, marker = "o")
plt.title ("Bresenham Line Drawing
algorithm (|m| < 1)")
plt.xlabel ("x-axis")
plt.ylabel ("y-axis")
plt.grid()
plt.show()
```

21/7/26

Result: The program was executed successfully Bresenham Line drawing Algorithm for $|m| < 1$.

Exp:2 Bresenham's Circle drawing Algorithm.

Aim: To implement Bresenham's circle drawing algorithm using the Python programming language.

Code:

```
import matplotlib.pyplot as plt
def Bresenham_circle(xc, yc, r):
    points = set()
    x = 0
    y = r
    def plot_symmetric_points(x_coord,
                              y_coord)
```

(xc + x, yc + y),

(xc - x, yc + y),

(xc + x, yc - y),

(xc - x, yc - y),

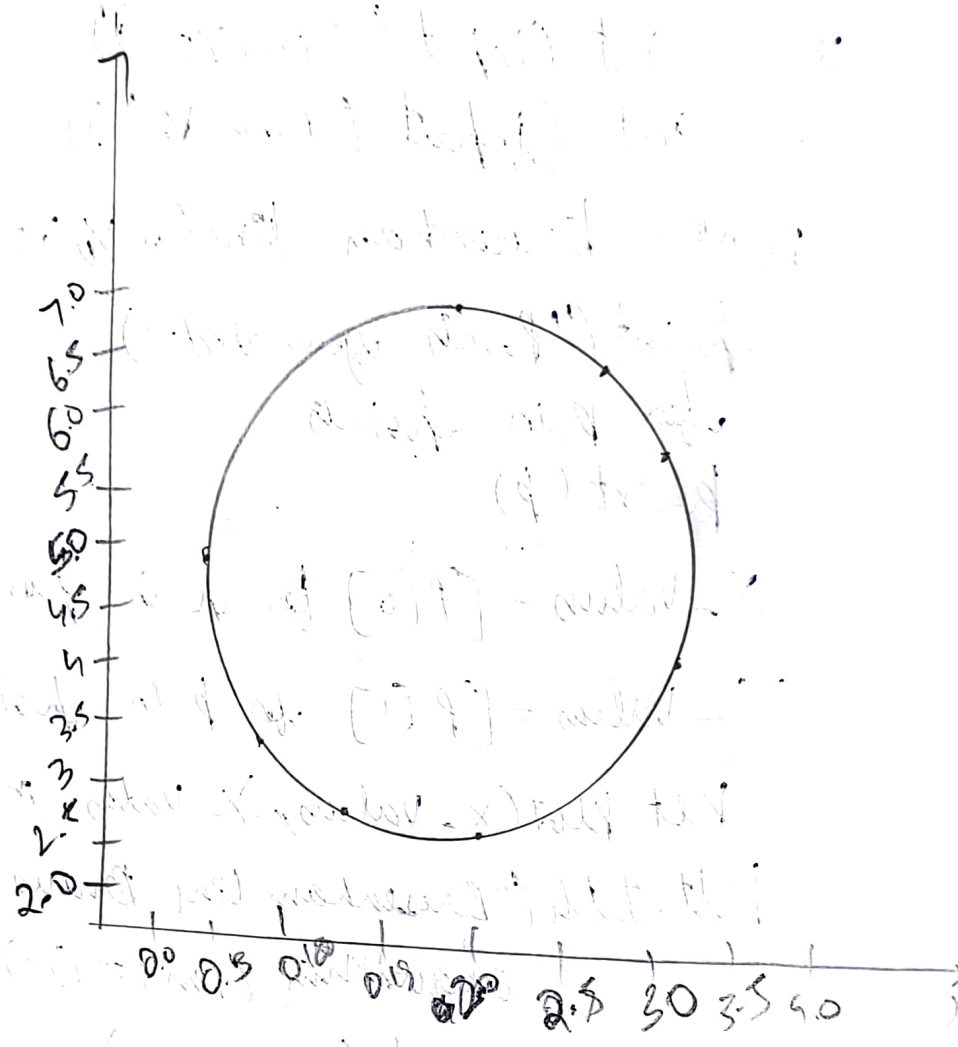
(xc - y, yc + x)

(xc - y, yc + x)

(xc - y, yc - x)

(xc - y, yc - x)

```
Pin points
d. append (P(0))
y. append [P[1]).
def bresenham_circle(x, xc, r):
```



Input: $x_c = 2$

$y_c = 2$

$Y = 4$

Points: (2,6), (2,6), (2,-2), (2,-2)
(6,2) (-2,2), (6,2)
(2,2), (6,2), (-2,2)
(3,6), (1,6), (3,-2)
(1,-2), (6,3), (4,5), (-2,1)

```
x = 0
y = r
p = 3 - 2r

x = []
y = []
while x <= y:
    plot_points(xc, yc, x, y, z, y)
    if p < 0:
        p = p + 4 * x + 6
    else:
        y -= 1
        p = p + 4 * (x - y) + 10
    x += 1
    return x, y
```

4/8/26

Result: Thus Bresenham circle drawing algorithm using python Programming is created.

Exp3: Bresenham Ellipse drawing algorithm.

Aim: To implement the Bresenham Ellipse drawing algorithm in Python.

Code:

```
import matplotlib.pyplot as plt
xc = int(input("enter x: "))
yc = int(input("enter x: "))
rxc = int(input("Radius x: "))
ryc = int(input("radius y: "))
x, y = 0, ry
rx2, ry2 = rx*rx, ry*ry
dx, dy = 0, 2*rx2*y
d1 = ry2 - rx2*ry + 0.25*rx2
x, y = [], []
def plot
x.extend([xc+x, xc-x, xc+x, xc-x])
y.extend([yc+y, yc+y, yc-y, yc-y])
```

while dx < dy

plot (x, y)

if $d_1 < 0$

$x++$

$dx += 2ry^2$

$d_1 += dx + ry^2$

else:

$x++$

$y--$

$dx += 2*ry^2$

$dy -= 2*rx^2$

$d_1 += dx - dy + ry^2$

$d_2 = ry^2 * (x+0.5)x * 2$

$+ rx + (y-1) * 2$

while $y > 0$

plot (x, y)

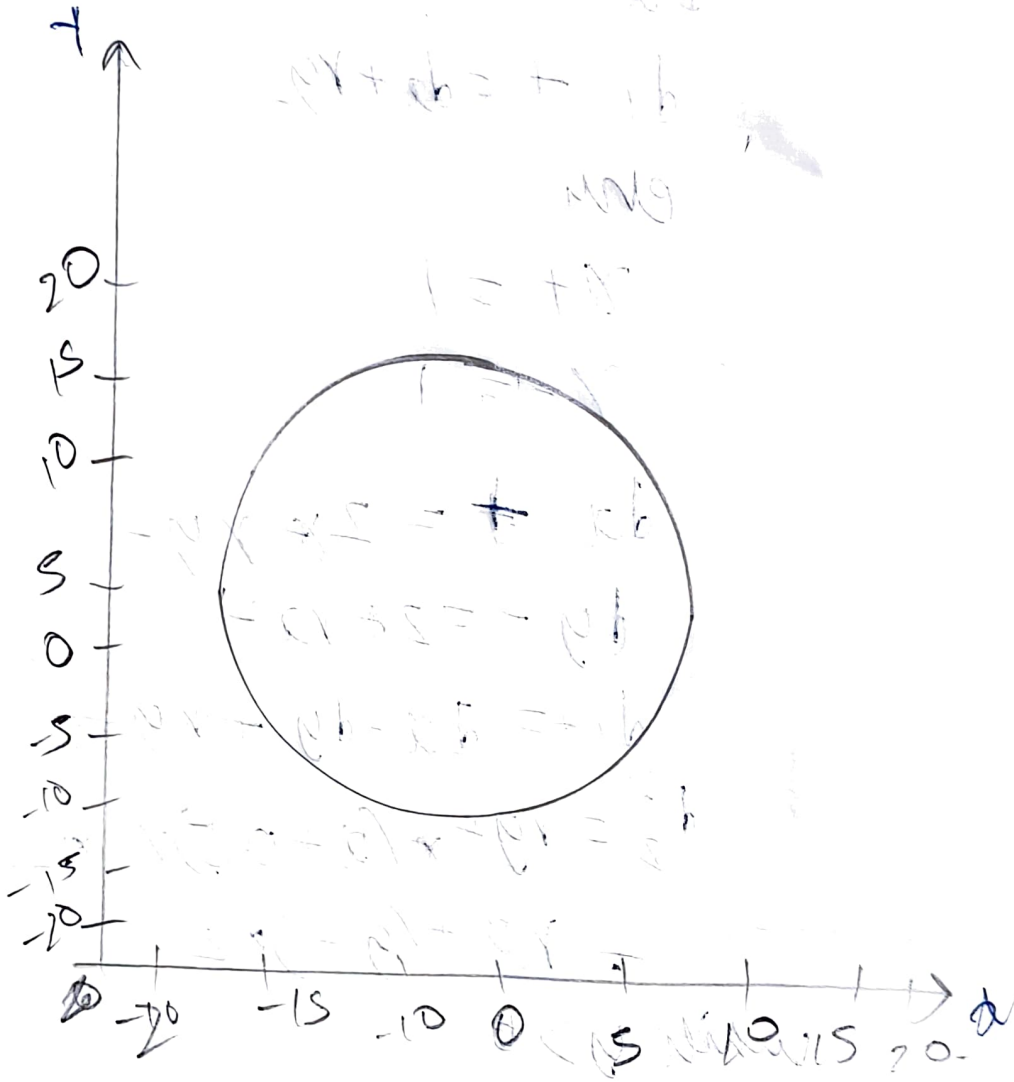
if $d_2 > 0$

$y--$

$dy -= 2*rx^2$

$d_2 += rx^2 - dy$

output:



Enter Center X: 0
Enter Center Y: 0
Enter x radius: 20
Enter y radius: 10.

```
em:
y-=1
x+=1
dx+=2*ry2
dy-=2+rx2
em=dx(-dy+rx2
    plt.scatter(x,y,c=10)
    plt.axis("equal")
    plt.grid()
    plt.title("Bresenham Ellipse").
    plt.show()
```

4/8/26

Result: Thus the implementation of Bresenham ellipse drawing Algorithm is verified Successfully

Exp: 4 DDA line drawing algorithm

Aim: To draw a dda line drawing algorithm using Python.

Code:

```
import matplotlib.pyplot as plt
def dda(x1, y1, x2, y2)
    dx = x2 - x1
    dy = y2 - y1
    steps = max(abs(dx), abs(dy))
    x_increment = dx / steps
    y_increment = dy / steps
        x = x1
        y = y1
        x_points = []
        y_points = []
    for i in range(steps + 1):
        x_points.append(round(x))
        y_points.append(round(y))
            x += x_increment
            y += y_increment
    plt.figure(figsize=(6, 6))
    plt.plot(x_points, y_points, color=
        linewidth=2, marker='o')
```

```
plt.title("DDA line drawing Algorithm")
plt.xlabel("X-Axis")
plt.ylabel("Y-Axis")
plt.grid(True)
plt.show()
x1 = int(input("Enter x1: "))
y1 = int(input("Enter y1: "))
x2 = int(input("Enter x2: "))
y2 = int(input("Enter y2: "))
dda(x1, y1, x2, y2)
plt.show()
```

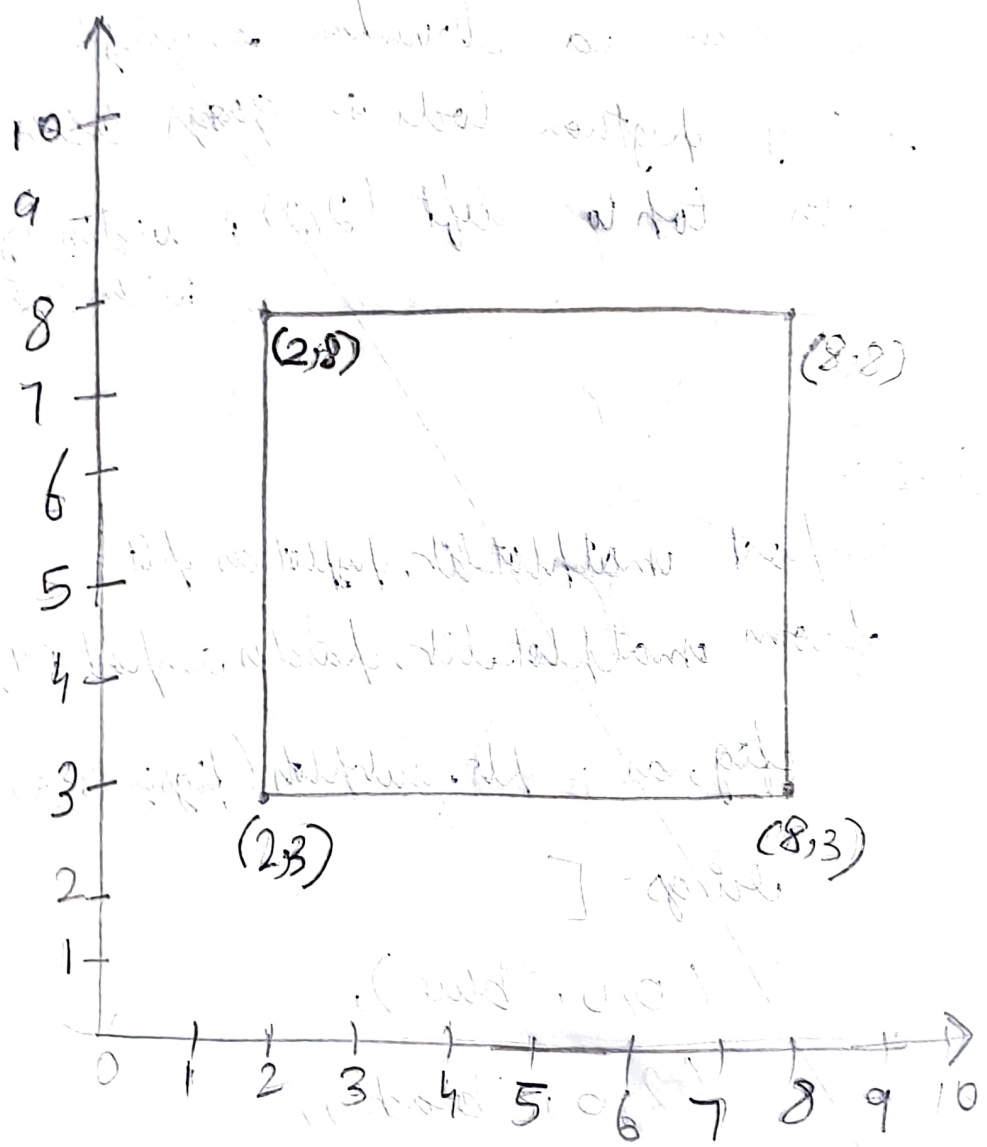
Results: DDA line drawing algorithm has been implemented.

Ex 5: Rectangle using Bresenham algorithm.

Aim: To draw a rectangle using Bresenham algorithm top to left (2,2) and width = 8, height = 5.

Code:

```
import matplotlib.pyplot as plt
def bresenham(x0, y0, x1, y1):
    points = []
    dx, dy = abs(x1 - x0), abs(y1 - y0)
    sx = 1 if x0 < x1 else -1
    sy = 1 if y0 < y1 else -1
    while True:
        points.append((x0, y0))
        if x0 == x1 and y0 == y1:
            break
        e = 2 * err
        if e > -dy:
            x0 += sx
```


(2,8)
(8,8)
(2,3)
(8,3)
0 1 2 3 4 5 6 7 8 9 10
1 2 3 4 5 6 7 8 9 10

```
if ye < dx
    en t = dx
    yo t = sy
    retun Points

x, y = 2, 2
w, h = 8, 5
p (bresenam (x, y, x+w, y) + bresenam
    (x+w, y, x+w, y+h) + bresenam
        (x+w, y+h, x, y+h),
    bresenam (x, y+h, x, y).
plt.grid()
plt.xlabel ("x4")
plt.title ("2D bresenam rectangle")
plt.show()
```

Results:

Thus the draw rectangle using Bresenham algorithm is implemented successfully.

Ex No 6: Bresenham Diagram
for traffic signal.

Aim: To draw traffic lights
using bresenhams algorithm.

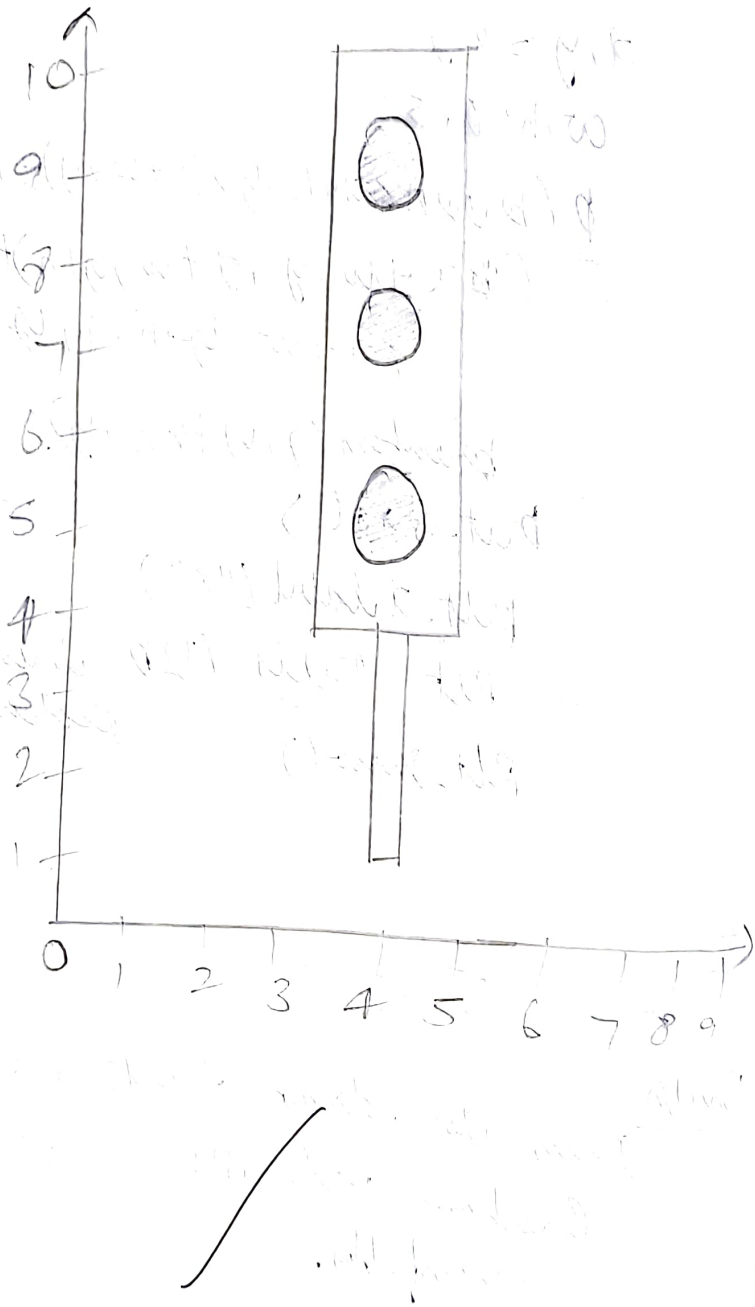
Code: import matplotlib.pyplot as plt.
from matplotlib.patches import
Circle, Rectangle fig, ax = plt.subplots()

ax.add_patch(Rectangle((2,1), 2, 6, color
edgecolor='black', linewidth=3,
alpha=0.8)).

ax.plot([3,3], [1,0], 'k', linewidth=6)
ax.set_xlim(0,8)
ax.set_ylim(0,8)
ax.axis('off')
plt.title("Traffic Signal")
plt.show.

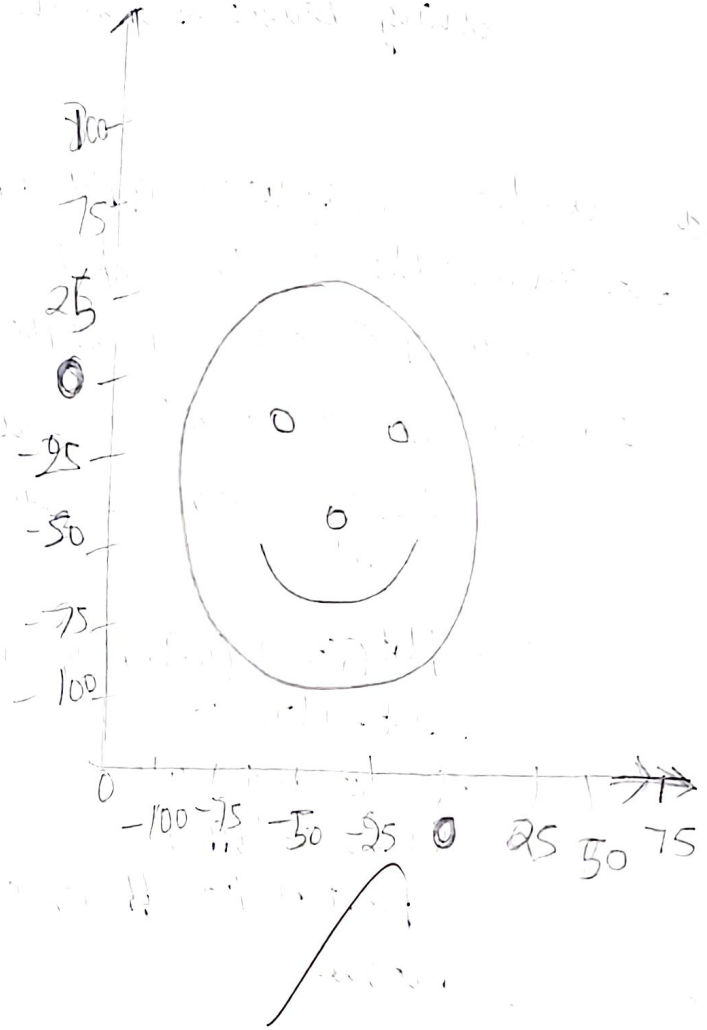
Result: Traffic lights are drawn
using bresenham.

Traffic Signal



Exp 7: Smiley face using Bresenham algorithm.

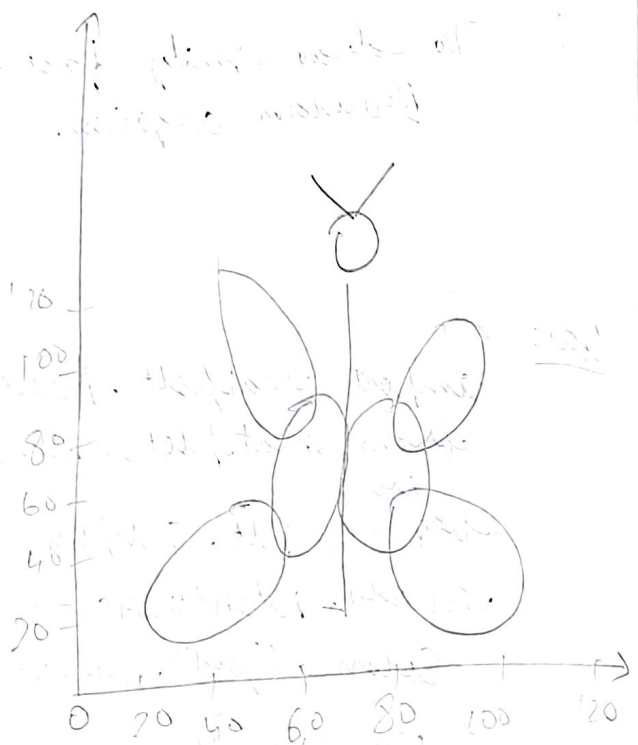
aim: To draw smiley face using Bresenham algorithm.



Code:

```
import matplotlib.pyplot as plt.
from matplotlib.patches import circle
fig, ax = plt.subplots(figsize=(6,6))
ax.add_patch(patch(circle((5,5),3,facecolor='gold'), edgecolor='black', linewidth=3))
ax.add_patch(patch(Circle((4,6),0.3,color='black'))
ax.add_patch(patch(Circle((4,6),0.3,color='black'))
ax.add_patch(patch(Circle((5,4),3),(theta1=180, theta2=360, color='black', linewidth=3))
plt.show
```

Results: Smiling face using bresenham algorithm is shown



Experiment-8.
Butterfly using Bresenham algorithm.

Aim: To draw butterfly using bresenham Algorithm.

code:

```
import turtle.
def draw_line (x1, y1, x2, y2):
    dx, dy = abs (x2-x1), abs (y2-y1)
    sx, sy = (1 if x1 < x2 else -1), (1 if y1 < y2 else -1)
    err = dx - dy

while true:
    turtle.penup(); turtle.goto(x1, y1); turtle.pendown();
    turtle.dot(3)
    if x1 == x2 & y1 == y2 : break
    e2 = 2 * err
    if e2 > -dy : err -= dy; x1 += sx.
    if e2 < dx : err += dx; x1 += sy.
turtle.speed (0); turtle.hideturtle().

lines = [(0,-50,0,50), (0,40,80,100), (80,100,120,80), (120,80,80,60), (80,60,0,40)]
for x1, y1, x2, y2 in lines:
    draw_line (x1, y1, x2, y2)
    draw_line (-x1, y1, -x2, y2).
turtle.done()
```

Result: Successfully drew the butterfly.

Experiment 9.

Olympic Logo using Bresenham Algorithm.

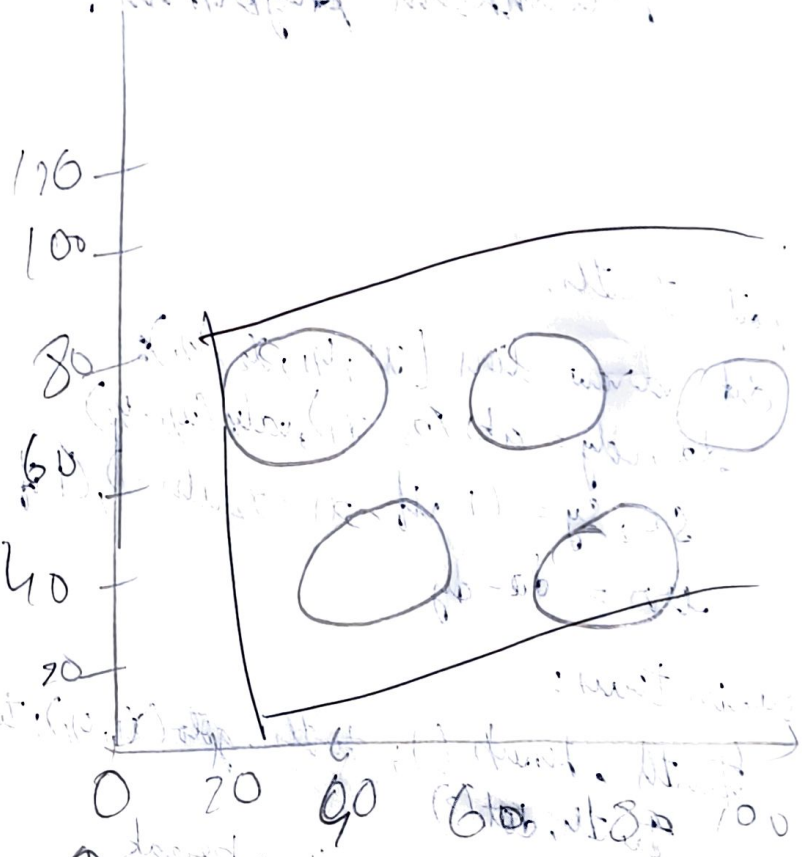
Aim: To draw an olympic logo using Bresenham Algorithm using python code.

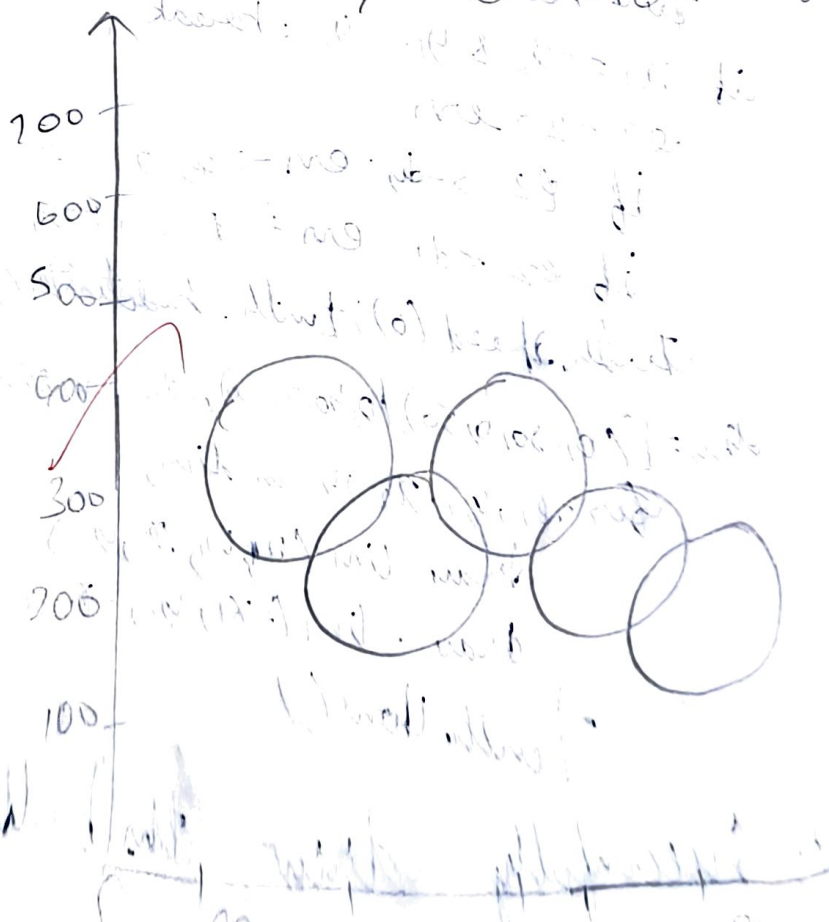
Code:

```
import matlab.pyplot as plt
import numpy as np.
def draw_ring(xc, yc, col):
for r in range (30, 38):
x, y, d = 0, r, 3-2*r
while y >= x:
    for dx, dy in [
    (x, y),
    (-x, y),
    (x, -y),
    (-x, -y),
    (y, x),
    (-y, x),
    (y, -x),
    (-y, -x),
```

```
]:
If 0 <= xc + dx < 400 and 0 <= yc
+dy < 250:
canvas [yc + dy, xc + dx] = col.
```

Output:





```
If   += 1
  if d>0:
    y-=1:
    d += 4*(x-y)+10
    else:
      d += 4*x*6
    canvas = np.ones((250,400,3))

circles=[
    (100,100,[0,0.5,0.8]),
    (140,140,[0.9,0.7,0.2])
    (180,100,[0,0,0])
    (220,140(0,0.6,0.3)).

    ]
plt.imshow(canvas)
    plt.axis("off")
    plt.show().
```

Result: Olympic logo has been created using bresenham algorithm

Experiment - 10

Monitor using Bresenham Algorithm

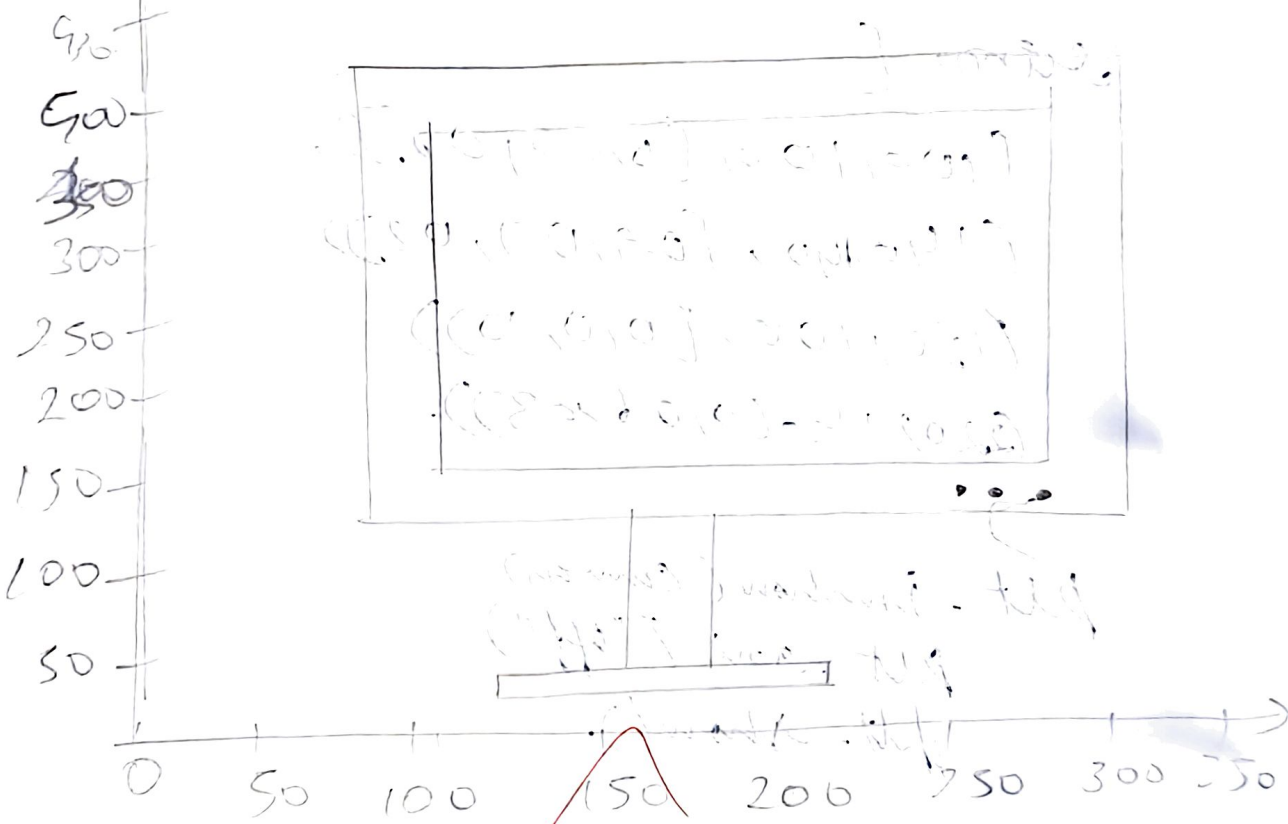
Aim: To draw monitor using bresenham algorithm.

Code:

```
import matplotlib.pyplot as plt
import numpy as np
img = np.ones((200,250,3))
def line(x1,y1,x2,y2,c=[0,0,0]):
    dx,dy = abs(x2-x1), abs(y2-y1)
    sx,sy = (1 if x1<x2 else -1),
      (1 if y1<y2 else -1)
    err = dx-dy.
while True:
if 0<=y1<200 & 0<=x1<250
img[y1,x1]=c.
if x1==x2 & y1==y2:
e2=2*err
if e2>-dy.
```

MONITOR USING BAYESIAN

ALGORITHM



line [40, 30, 210, 30)
line (210, 30, 210, 130)
line (210, 130, 40, 130)
line (40, 130, 40, 30)

line (40, 38, 202, 38)
line (202, 38, 202, 52)
line (202, 52, 48, 52)
line (48, 52, 48, 38)

plt.imshow(img)
plt.axis("off")
plt.show().

Results: Monitor has been drawn using Bresenham algorithm.

Experiment - 11

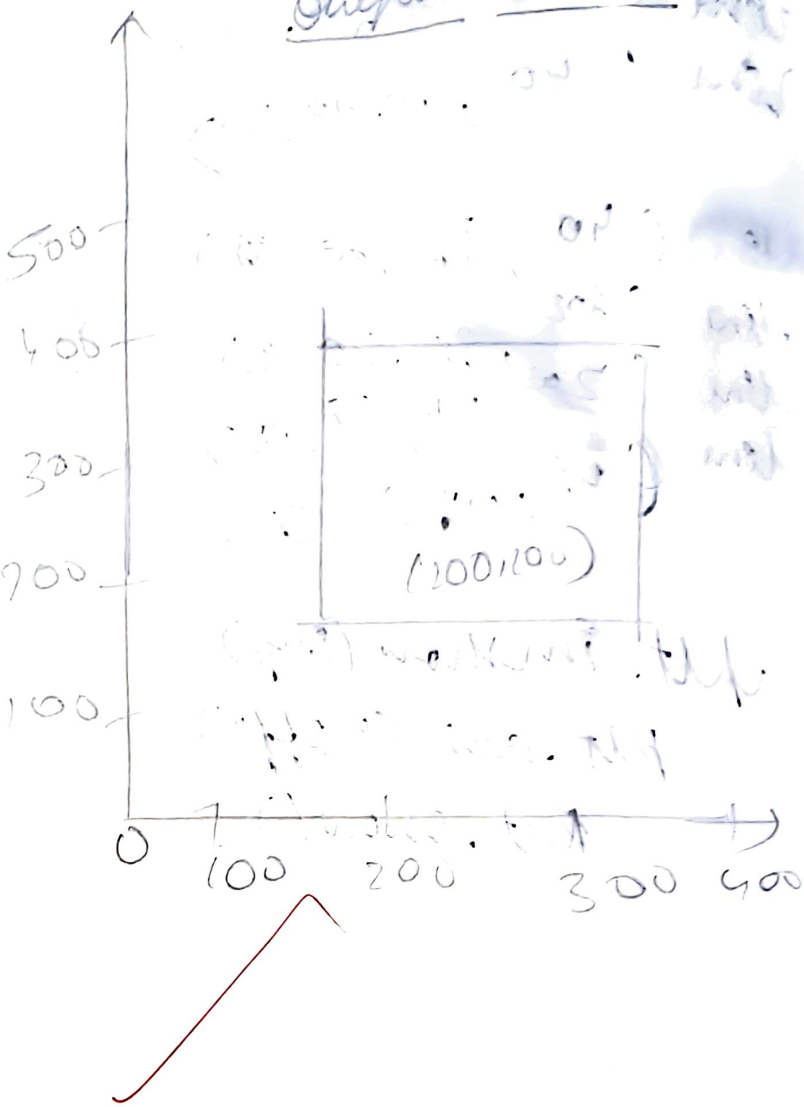
Flood fill algorithm.

Aim: To implement flood fill algorithm using scan fill algorithm.

```
def flood_fill (grid, r, c, new_colour):
    orig = grid[r][c]
    if orig == new_colour:
        return grid
    def fill (x, y):
        if (
            0 <= x < len (grid)
            and 0 <= y < len (grid[0])
        ):
            grid[x][y] = new_colour
            fill (x+1, y)
            fill (x-1, y)
            fill (x, y+1)
            fill (x, y-1)
    fill (r, c)
    for row in grid:
        print (row):
```

Result: flood fill algorithm has been implemented.

Output Image



Experiment - 12

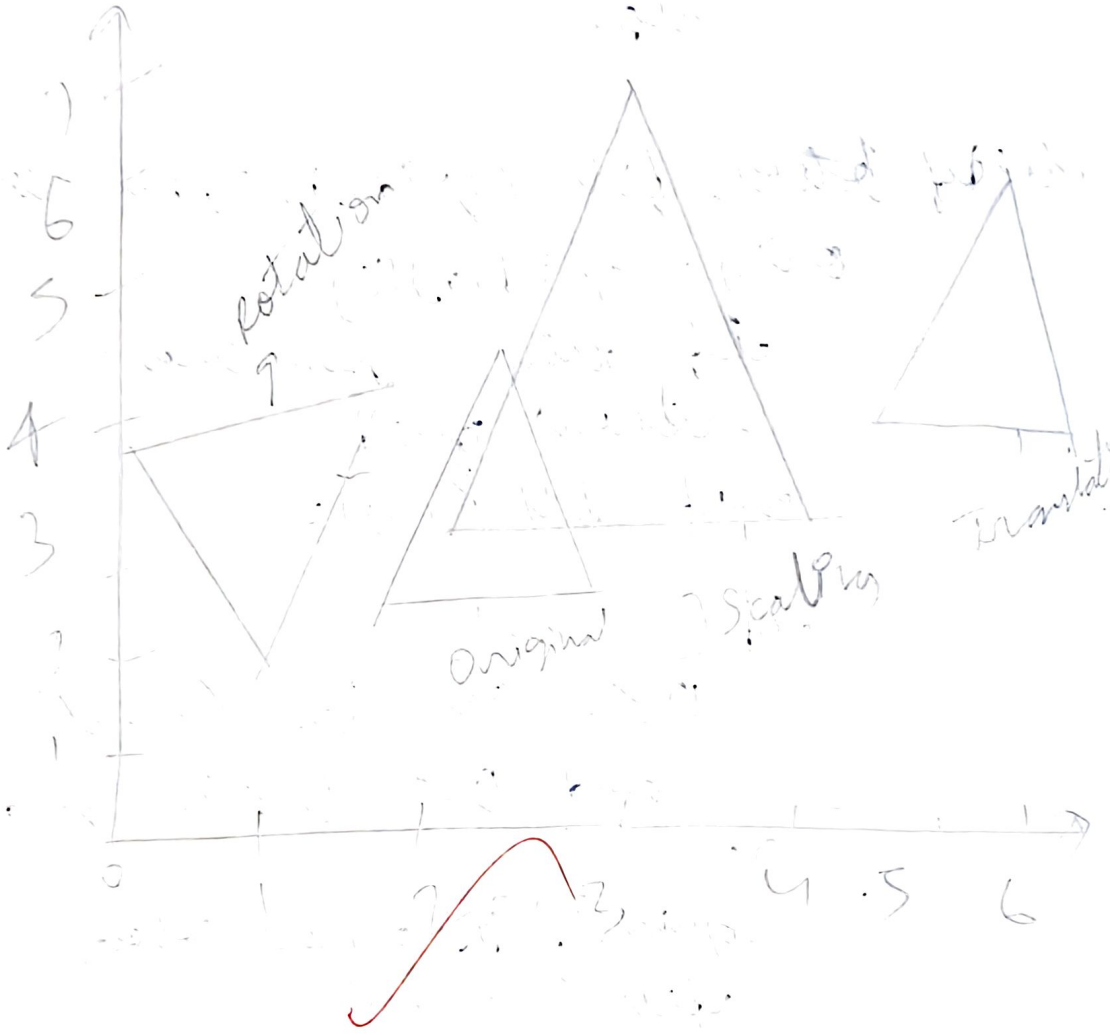
Basic 2D transformations,

Aim: To implm basic 2D transformations using python code.

Code:

```
import numpy as np
import matplotlib.pyplot as plt
p = np.array([[1,1],[3,1],[3,3],
              [1,3]])
t = p + [4,1]
s = p*1.5
a = np.radians(45)
R = np.array([[np.cos(a),
    -np.sin(a),np.cos(a)]])

plt.plot(p[:,0],p[:,1],b)
plt.plot(t[:,0],t[:,1],r)
plt.plot(s[:,0],s[:,1],g)
plt.plot(r[:,0],r[:,1],m)
```


rotation
Original
Scaling
Translation
0
1
2
3
4
5
6

```
label = "original")
label = 'Translation')
label = ('Scaling")
label = ("Rotation")

plt.grid()
plt.axis('equal")
plt.xlabel("x")
plt.ylabel("y")
plt.title("2D transformation")
plt.legend()
plt.show().
```

~~Output:~~

Result: 2D transformation diagram shown then drawn.

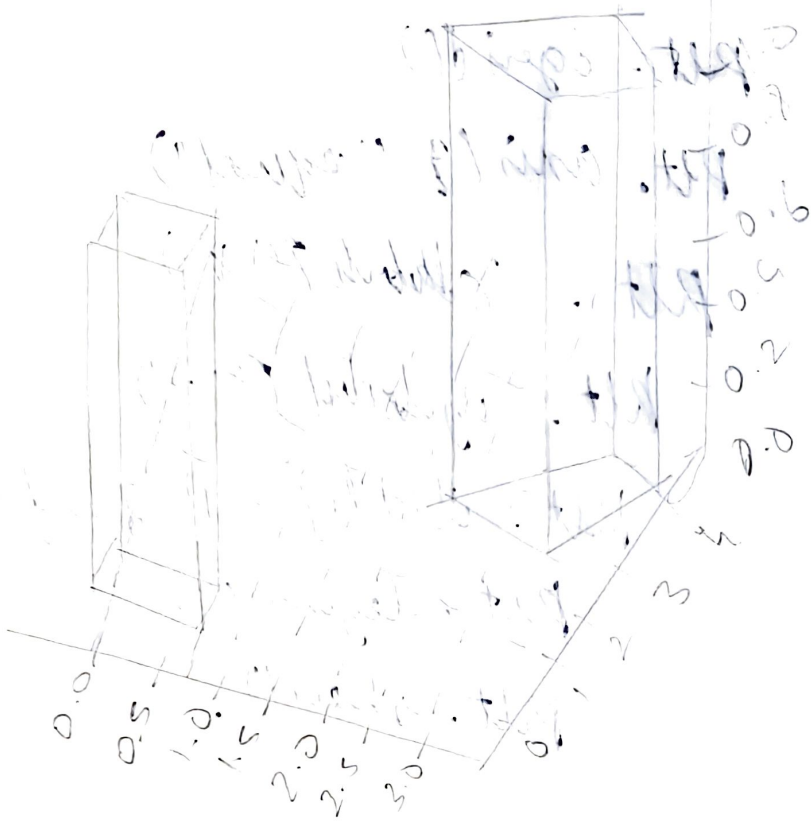
Experiment - 13

CLIP POLYGONS CLIPPING USING SUTHERLAND- HODGMAN ALGORITHM

Aim: To write a program that clips a specified rectangular window using sutherland hodgman polygon clipping

Program:

```
import matplotlib.pyplot as plt
LEFT, RIGHT, BOTTOM, TOP = 0, 1, 2, 3.
def inside (p, edge, clip_win):
    x, y = p.
    xmin, xmax, ymin, ymax = clip_win.
    if edge == LEFT:
        return x >= xmin.
    elif edge == BOTTOM:
        return y >= ymin
    elif edge == TOP:
        return y <= ymax.
def intersect (p1, p2, edge, clip_win):
```



```
xmin, xmax, ymin, ymax = clip_win
x1, y1 = p1
x2, y2 = p2
if edge == LEFT:
    x = xmin
    y = y1 + (y2 - y1) * (xmin - x1) / (x2 - x1)
elif edge == RIGHT:
    x = xmax
    y = y1 + (y2 - y1) * (xmax - x1) / (x2 - x1)
elif edge == BOTTOM:
    y = ymin
    x = x1 + (x2 - x1) * (ymin - y1) / (y2 - y1)
elif edge == TOP:
    y = ymax
    x = x1 + (x2 - x1) * (ymax - y1) / (y2 - y1)
return (x, y)

draw_polygon(clipped_poly, 'red',
             "Clipped Polygon")

plt.legend()
plt.title("Polygon clipping using
           Sutherland-Hodgman
           algorithm")
plt.grid(True)
plt.axis("equal")
plt.show()
```

Result: The polygon was successfully clipped using the Sutherland-Hodgman algorithm against a rectangular clipping window.

Experiment - 14

3-D transformations on Basic Objects (Cube, Pyramid).

Aim: To create a program that allows the user to perform 3D transformations on the a cube or pyramid & visualize the results.

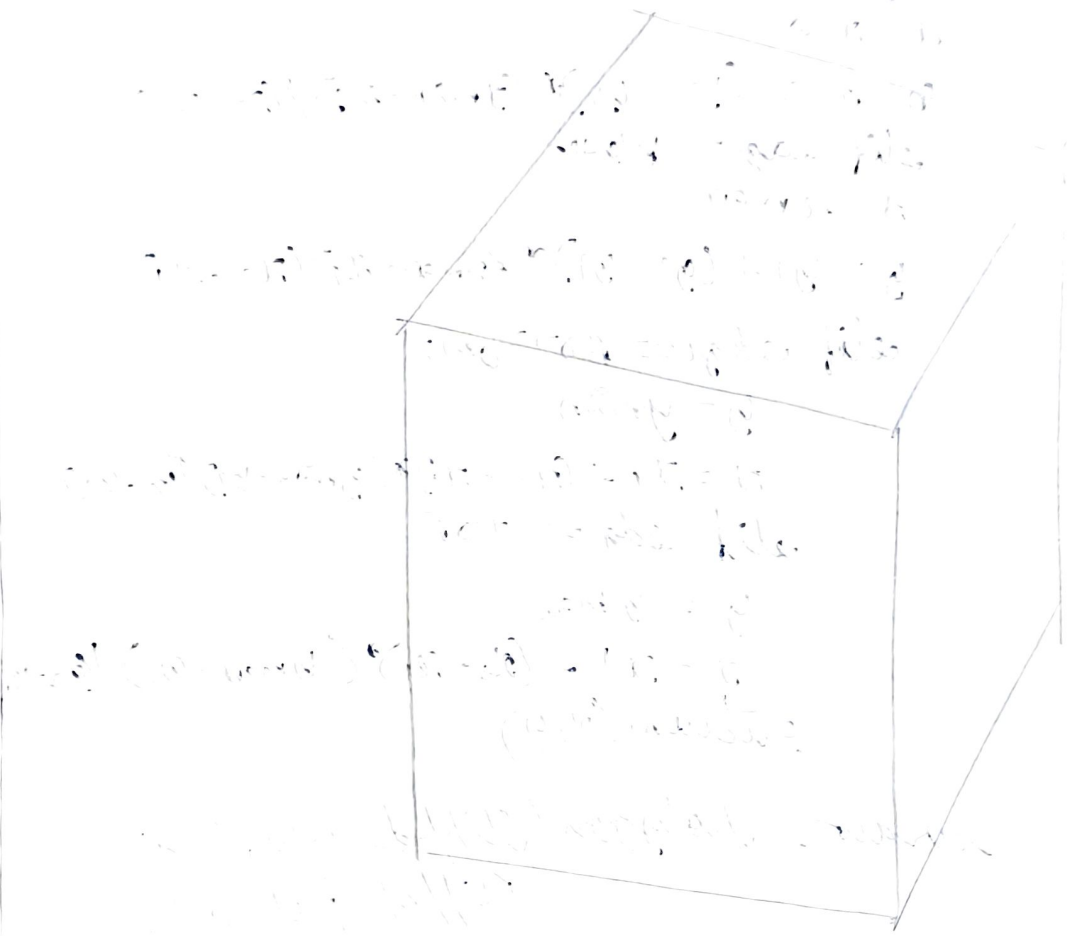
Program:

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d.art3d.

def draw_edges(ax, vertices, edges, color='b'):
    lines = [(vertices[start], vertices[end]) for
    start, end in edges]

def translation_matrix(tx, ty, tz):
    return np.array([[1,0,0,tx],
                     [0,1,0,ty],
                     [0,0,1,tz],
                     [0,0,0,1]])
```

3d Cube with flat shading.



```
def apply_transform(vertices, matrix):
    transformed = []
    for v in vertices:
        vec = np.array([*v, 1])
        result = matrix @ vec
        transformed.append(result[:3])
    return transformed

# Define cube
Vertices = [(0,0,0), (1,0,0), (1,1,0), (0,1,0),
            (0,0,1), (1,0,1), (1,1,1)]

edges = [(0,1), (1,2), (2,3), (3,0),
         (4,5), (5,6), (6,7), (7,4),
         (0,4), (1,5), (2,6), (3,7)]

# Apply transformations
T = translation_matrix(2,2,6)
S = scaling_matrix(1.5,1.5,1.5)
R = rotation_matrix_z(45)

ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')
plt.show()
```

Result: The code was executed to perform translation, scaling and rotation on a 3D cube.

Experiment - 15.

Create & Render 3D objects Using Polygons and Apply Basic colour and Shading Techniques.

Aim: To create and Render 3D objects using polygonal modeling.

Program:

```
import matplotlib.pyplot as plt.
from mpl_toolkits.mplot3d.art3d
    import Poly3DCollection
import numpy as np
vertices = np.array([[0,0,0],[1,0,0],
                     [1,1,0],[0,1,0]
                     [1,1,1],[0,1,1]])

faces = [[vertices[j] for j in [0,1,2,3].
[vertices[j] for j in [4,5,6,7]],
[vertices[j] for j in [0,1,5,4]],
[vertices[j] for j in [2,3,7,6]],
[vertices[j] for j in [1,2,6,5]],
```

Result: 3D polygonal objects were successfully created and rendered. Basic color & flat shading is applied.

Experiment-16

Create a multimedia application that integrates, Image, Sound & Video in a simple user interface.

Aim: To develop a simple multimedia application that integrates images, audio & video into a unified user interface using Python.

Program:-

```
import tkinter as tk
from tkinter import filedialog
from PIL import Image, ImageTk
from playsound import playsound
import cv2
import threading

def play_audio():
    playsound('example_audio.mp3')

def play_video():
    cap = cv2.VideoCapture('sample_
                           video.mp4')

while mp4.isOpened():
if not ret:
    break
cv2.imshow("Video", frame)
```

```
if (cv2.waitKey(25) & 0xFF == ord('q')):
    break.
cap.release()
cv2.destroyAllWindows()
def load_image():
    img = Image.open('sample_image.jpg')
    img = img.resize((300, 300))
    img_tk = ImageTk.PhotoImage
    panel.configure
    panel.image = img_tk.
btn_img = tk.button.
btn_audio = tk.button.
    btn_video = tk.button.
panel = tk.Label
    panel.pack()
    btn_img.pack(pady=5)
    btn_audio.pack(pady=5)
    btn_video.pack(pady=5)
window.mainloop()
```

Result: A multimedia application was successfully created that integrates image, audio & video functionalities into a user interface.

Experiment - 17

Capturing Video/Audio from webcam or Microphone & Display on multimedia Interface.

Aim: To develop a program that captures live video from the webcam & live audio from the microphone.

Program:

```
import cv2
import scipy.io.wavfile import write
import threading
import tkinter as tk.
# Audio recording function
    def record_audio():
        duration = 5 # seconds
        fs = 44100 # Sampling rate.
        print ("Recording Audio")
        audio = sd.rec (int(duration*fs)
        sd.wait()
        write ('recorded.wav')
        def capture function.
```

Result: Live video and audio were successfully captured using webcam & microphone & displayed through a multimedia interface.

Experiment-17

Capturing Video/Audio from webcam or Microphone & Display on multimedia Interface.

Aim: To write a program that captures live video from the webcam & live audio from the microphone.

Program:

```
import cv2
import Scipy.io.wavfile import write
import threading
import tkinter as tk.
#Audio recording function
def record_audio():
duration = 5# Seconds
fs = 44100 # Sampling rate.
print ("Recording Audio")
audio = sd.rec (int(duration*fs)
sd.wait()
write ('recorded.wav')
def capture function.
```

Result: Live video and audio were successfully captured using webcam & microphone & displayed through a multimedia interface.

Experiment - 18

Create Application to Recive or send texts.

Aim: To create application to Recive or send texts.

Program:

```
import socket
import threading.
s = socket.socket()
s.connect(("127.0.0.1", 12345))

def recive():
 while True:
  try:
   print("\n Friend:", s.recv(1024).d
  except:
   break.
threading.Thread(target=recive, daem
        .Start()
while True:
 msg = Input("You:")
 s.send(msg.encode())
```

Result: An application to Recive or send texts have been create